

# Matris ile Kodlama — Kodların Mantığı

Oct 8, 2026 · @Arif

## 1. Büyük resim: neden matris.h var?

Öğrencinin kodu LED'lere hiçbir zaman doğrudan dokunmaz. Sadece bir "ekran hafızası"nı değiştirir. `matris.h` içindeki zamanlayıcı ise saniyede 2500 kez bu hafızaya bakar ve LED'leri sırayla yakar.

**Problem: 64 LED, 16 pin**

Matrisin içinde her LED bir satır teli ile bir sütun telinin kesiştiği yerde durur. 64 LED'i 16 pinle kontrol edebilmemizin sebebi bu. Bedeli de şu: **aynı anda farklı satırlara farklı desen veremezsin.**

Örnek: (satır 1, sütun 3) ve (satır 2, sütun 5) yansın istiyorsun. Satır 1 ve 2'yi açıp sütun 3 ve 5'i açarsan, istemediğin (1, 5) ve (2, 3) de yanar. Çünkü açık her satır, açık her sütunla kesiştiği yerde LED yakar.

**Çözüm: çoklama (multiplexing)**

Aynı anda sadece **bir satırın küçük bir parçasını** yakarız, sonra onu söndürüp sıradakine geçeriz. Bunu saniyede yüzlerce kez yaparsak göz bütün resmi aynı anda yanıyor sanır. Buna "görme sürekliliği" denir; sinemanın çalışma mantığı da budur. Saniyede yaklaşık 50 kereden hızlı yenilenen bir görüntüyü göz titremeden görür.

**Stacker'daki yöntem ile yenisi arasındaki fark**

|                                         | Stacker kodu                                                    | matris.h                                                                                    |
|-----------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Yenilemeyi kim yapıyor?                 | Ana program, <code>goster(ms)</code> fonksiyonu içinde          | Arka planda çalışan zamanlayıcı kesmesi (Timer1)                                            |
| <code>delay()</code> kullanılabilir mi? | Hayır. <code>delay</code> sırasında yenileme durur, ekran donar | Evet. Kesme <code>delay</code> sırasında da çalışır                                         |
| Öğrenci ne yazıyor?                     | Oyun mantığı + her yerde <code>goster()</code>                  | Sadece <code>ledYak</code> , <code>resimCiz</code> , <code>delay</code> gibi kolay komutlar |

## Dosyalar birbirine nasıl bağlanıyor?

Her proje klasöründe iki dosya var: `pX_...ino` ve `matris.h`. Arduino IDE aynı klasördeki `.h` dosyasını yan sekmede açar. `.ino` dosyasının en üstündeki `#include "matris.h"` satırı, derleyiciye "önce bu dosyanın içeriğini buraya yapıştır" der. Tırnak işareti ( `"..."` ) önce sketch'in kendi klasörüne bakılmasını sağlar. Bu yüzden `matris.h` her klasörde ayrı kopya olarak durur.

Not: Normalde `.h` dosyasına fonksiyonun gövdesi yazılmaz, sadece adı yazılır. Aynı `.h` iki farklı `.cpp` dosyasına dahil edilirse "çift tanım" hatası çıkar. Tek dosyalık bir Arduino sketch'inde bu sorun yaşanmadığı için, işi basit tutmak adına her şeyi tek dosyada bıraktım.

### Öğrenci sadece hafızayı değiştirir, LED'leri kesme yakar

#### ANA PROGRAM (setup / loop)



matris.h'nin iki katmanı · ana program ve kesme

Üst sıra senin ve öğrencinin yazdığı kodla çalışır, alt sıra arka planda kendi kendine döner. İkisinin bulunduğu tek yer ekran hafızasıdır; dokümanın geri kalanı bu kutuları tek tek açıyor.

## 2. Donanım: matrisin içinde ne var?

Bir LED'in yanması için satır pini HIGH, sütun pini LOW olmalı. 1088BS matriste satırlar LED'lerin artı ucuna (anot), sütunlar eksi ucuna (katot) bağlı.

## Bir LED nasıl yanıyor?

| Satır pini | Sütun pini | LED                                |
|------------|------------|------------------------------------|
| HIGH (5 V) | LOW (0 V)  | Yanar: akım anottan katoda akar    |
| LOW        | HIGH       | Söner: LED ters yönde, akım geçmez |
| HIGH       | HIGH       | Söner: iki uç aynı gerilimde       |
| LOW        | LOW        | Söner                              |

Kodda sıkça gördüğün `SATIR_ANOT ? HIGH : LOW` ifadesi "üçlü operatör"dür. Şu anlama gelir: `SATIR_ANOT` true ise HIGH, değilse LOW kullan. Matrisin tipi farklıysa (ortak katot) tek bir ayarı değiştirmek tüm kodu ters çevirir.

## Pin listeleri neden karışık?

```
const byte _SATIR[8] = {A3, 12, 9, A0, 2, 8, 3, 6};
const byte _SUTUN[8] = {13, 4, 5, A2, 7, A1, 11, 10};
```

Matrisin 16 bacağı satır ve sütunlara sıralı değil, karışık dağılmış durumda. Kabloları breadboard'a sırayla taktık, karışıklığı bu iki liste çözüyor. `_SATIR[0] = A3` şu demek: matrisin 1. satırı Arduino'nun A3 pinine bağlı. Kod "1. satırı yak" dediğinde listeden A3'ü bulup onu HIGH yapar.

## Dirençler neden sütunlarda?

LED'in kendi direnci neredeyse sıfırdır; dirençsiz bağlarsan çok akım çeker ve LED de, pin de zarar görür. 220  $\Omega$  direnç akımı sınırlar:

$$\text{Akım} = (5 \text{ V} - 2 \text{ V}) / 220 \Omega \approx 13 \text{ mA}$$

Buradaki 2 V, kırmızı LED'in üzerinde düşen gerilimdir. Pratikte Arduino pininin kendi iç direnci yüzünden akım LED başına 9–12 mA civarında kalır.

Dirençler sütunlarda olduğu için, **bir satırdaki her LED'in kendi direnci var**. Bu yüzden kodu "bir anda bir satır" mantığıyla kurduk. Tersini yapsaydık, yani bir sütunu bütün satırlarla yaksaydık, 8 LED tek direnci paylaşır ve hepsi sönük yanardı.

## Neden aynı anda en fazla 3 LED?

Satır pinlerinde direnç yok. Bir satırdan N tane LED yanarsa o satır pini  $N \times \sim 10 \text{ mA}$  akım vermek zorunda kalır. Arduino pininin mutlak sınırı 40 mA, tavsiye edilen değer 20 mA.

| Aynı anda yanan LED (tek satırda) | Satır pininden geçen akım | Durum                      |
|-----------------------------------|---------------------------|----------------------------|
| 1                                 | ~10 mA                    | Güvenli                    |
| 3 (bizim kod)                     | ~25–30 mA                 | Sınırın altında            |
| 8 (satırın tamamı)                | ~70–100 mA                | Tehlikeli, pin bozulabilir |

Bu yüzden her satırı 3 parçaya bölüyoruz: sütun 0–2, 3–5 ve 6–7. Bu parçaların kodda nasıl kullanıldığı 3. ve 5. bölümde anlatılıyor.

### 3. Ekran hafızası: byte, bit ve bit maskesi

64 LED'in durumu sadece 8 byte'ta saklanır: her satır için bir byte, o byte'ın her biti bir LED. Stacker kodunda `bool ekran[8][8]` vardı ve 64 byte tutuyordu. Bit kullanmak hem 8 kat az RAM harcıyor hem de 5. bölümdeki taramayı tek işlemle yapmayı sağlıyor.

#### Byte ve bit

Bir `byte` 8 bitten oluşur ve 0–255 arası bir sayı tutar. `_ekran[y]` satırın byte'ıdır; onun `x`. biti (`x, y`) noktasındaki LED'dir. Bit 1 ise LED yanık, 0 ise sönmük.

```
byte _ekran[8];           // _ekran[3] = 3. satırın 8 LED'i
_ekran[3] = 0b00010010;  // bit 1 ve bit 4 açık -> (1,3) ve (4,3) yanar
```

Kafa karıştıran nokta: ikilik sayı yazılırken **bit 0 en sağdadır**, ama ekranda `x = 0` en soldaki LED. Yani `0b00010010` yazdığını sağdan sola okursun: bit0 = 0, bit1 = 1, bit2 = 0, bit3 = 0, bit4 = 1 ... Bu sadece yazım alışkanlığı, LED'in yeri değişmiyor.

#### Tek bir biti seçmek: `1 << x`

`<<` "sola kaydır" işlemidir. `1 << x`, 1 sayısını `x` kez sola kaydırır; sonuçta sadece `x`. biti 1 olan bir sayı oluşur. Buna **maske** denir.

| x | 1 << x (ikilik) | Ondalık |
|---|-----------------|---------|
| 0 | 00000001        | 1       |
| 1 | 00000010        | 2       |
| 2 | 00000100        | 4       |
| 3 | 00001000        | 8       |
| 7 | 10000000        | 128     |

## Üç temel işlem

| Ne yapıyor | Kod                                        | Mantık                                                                                   |
|------------|--------------------------------------------|------------------------------------------------------------------------------------------|
| Yak        | <code>satir  = (1 &lt;&lt; x);</code>      | VEYA ( ): maskede 1 olan bit kesin 1 olur, diğerleri aynı kalır                          |
| Söndür     | <code>satir &amp;= ~(1 &lt;&lt; x);</code> | ~ maskeyi tersine çevirir (sadece x. bit 0 olur). VE (&) o biti 0'lar, diğerlerini korur |
| Oku        | <code>satir &amp; (1 &lt;&lt; x)</code>    | Sonuç 0 değilse x. bit açık, yani LED yanık                                              |

Sayılarla örnek, x = 4 için:

```

satir          = 0b000000010 // sadece (1,y) yanık
1 << 4         = 0b00010000
satir | maske   = 0b00010010 // yaktık: artık (1,y) ve (4,y) yanık
~(1 << 4)       = 0b11101111
satir & ~maske  = 0b000000010 // söndürdük: geri döndü

```

`ekraniDoldur()` her satırı 255 yapar ( `0b11111111` , 8 bitin hepsi açık). `ekraniTemizle()` her satırı 0 yapar.

## Parça maskeleri: `_PARCA`

```
const byte _PARCA[3] = {0x07, 0x38, 0xC0};
```

`0x` ile başlayan sayılar on altılık (hex) yazımdır. Her hex basamak 4 bite karşılık gelir, bu yüzden bit maskelerini kısa yazmak için kullanılır.

| Maske             | İkilik   | Hangi sütunlar |
|-------------------|----------|----------------|
| <code>0x07</code> | 00000111 | 0, 1, 2        |

| Maske | İkilik   | Hangi sütunlar |
|-------|----------|----------------|
| 0x38  | 00111000 | 3, 4, 5        |
| 0xC0  | 11000000 | 6, 7           |

`_fiziksel[r]` & `0x38` yazınca satırın sadece 3–5. sütunlarındaki bilgi kalır. Tek bir işlemle "bu parçada hangi LED'ler yanacak?" sorusunun cevabını alırız.

İki hafıza: `_ekran` ve `_fiziksel`

- `_ekran[y]` : öğrencinin gördüğü yön.  $y = 0$  en alt satır,  $x = 0$  en sol sütun.
- `_fiziksel[r]` : matrisin pin sırasına göre.  $r$  = satır pini numarası, bit  $c$  = sütun pini numarası. Döndürme ve ayna burada uygulanmış haldedir.

İkisini ayrı tutmamızın sebebi hız. Döndürme hesabı sadece LED değiştiğinde bir kez yapılır. Saniyede 2500 kez çalışan kesme ise hazır `_fiziksel` dizisini okur, hiçbir hesap yapmaz.

`volatile` ne demek?

```
volatile byte _fiziksel[8];
```

Derleyici hız kazanmak için bir değişkenin değerini bazen bir kenarda (önbellekte) tutar ve bellekten tekrar okumaz. `volatile` derleyiciye "bu değişkeni kesme fonksiyonu da kullanıyor, her seferinde gerçek değerini bellekten oku" der. Ana program ile kesmenin paylaştığı değişkenlerde kullanılır.

Titremeyi önlemek: `_ekranaYaz`

```
void _ekranaYaz(const byte yeni[8]) {
    byte f[8];
    _fizikseliHesapla(yeni, f); // önce yeni fiziksel resmi hazırla
    noInterrupts();           // kesmeyi kısa süre durdur
    for (byte i = 0; i < 8; i++) { _ekran[i] = yeni[i]; _fiziksel[i] = f[i]; }
    interrupts();             // kesmeyi tekrar aç
}
```

Bütün komutlar önce bir **kopya** ( `yeni` ) üzerinde çalışır, sonra hazır resmi tek seferde gerçek hafızaya yazar. Kopyalama sırasında kesme kapatılır.

Neden? `resimCiz` önce ekranı silip sonra yeni resmi çizseydi, kesme tam aradaki boş anı yakalayabilirdi. O zaman ekran bir anlığına söner ve titreme görülürdü. Kopyalama 8 byte olduğu için birkaç mikrosaniye sürer; kesmenin bu kadar gecikmesi fark edilmez.

## 4. Koordinat dönüşümü: `_cevir`

`_cevir`, öğrencinin (x, y) noktasını matrisin (satır pini, sütun pini) çiftine çevirir. Döndürme ve ayna ayarları burada uygulanır.

```
void _cevir(byte x, byte y, byte &satir, byte &sutun) {
    if (!_ayna) x = 7 - x;
    for (byte k = 0; k < _donusAdimi; k++) {
        byte eski = x;
        x = y;
        y = 7 - eski;
    }
    sutun = x;
    satir = 7 - y;
}
```

### Adım 1: Döndürme yokken

`sutun = x` ve `satir = 7 - y`. Sütunlar zaten soldan sağa sıralı. Satır pinlerinde ise 0 numara matrisin **en üst** satırı. Öğrenci için  $y = 0$  en alt olsun istedik (matematikteki koordinat düzlemi gibi), bu yüzden 7'den çıkarıyoruz:  $y = 0 \rightarrow \text{satır pini } 7$ ,  $y = 7 \rightarrow \text{satır pini } 0$ .

### Adım 2: 90 derece döndürme

Her döngü adımı noktayı 90° çevirir. Kural: yeni x = eski y, yeni y = 7 - eski x. Köşeleri izleyerek kontrol edebilirsin; her adımda bir sonraki köşeye geçerler:

| Başlangıç      | 1 adım (90°)   | 2 adım (180°)  | 3 adım (270°)  | 4 adım            |
|----------------|----------------|----------------|----------------|-------------------|
| (0, 0) sol alt | (0, 7) sol üst | (7, 7) sağ üst | (7, 0) sağ alt | (0, 0) başa döndü |

`eski` değişkeni neden var? `x = y;` satırı x'in eski değerini siler. Ama bir sonraki satırda eski x'e ihtiyacımız var. Bu yüzden önce onu `eski` adlı geçici bir kutuya koyuyoruz. Bu, programlamadaki klasik "iki değeri yer değiştirme" (swap) tekniğidir.

### Adım 3: Ayna

`x = 7 - x` soldaki noktayı sağa, sağdakini sola taşır. Matris breadboard'a ters yüzle takılmış gibi görünüyorsa işe yarar.

## &satir ve &sutun ne?

Bir fonksiyon `return` ile sadece tek değer döndürebilir, bizim ise iki sonuca (satır ve sütun) ihtiyacımız var. `&` işareti "referans" demektir: fonksiyon, çağıranın değişkenine doğrudan yazar. `_cevir(x, y, r, c);` çağırısından sonra `r` ve `c` dolmuş olur.

## Derece nasıl adıma çevriliyor? ( `matrisBaslat` içinde)

```
derece = derece % 360; // 450 -> 90
if (derece < 0) derece += 360; // -90 -> 270
_donusAdimi = ((derece + 45) / 90) % 4; // en yakın 90'ın katına yuvarla
```

| Yazılan | % 360 sonrası | (d + 45) / 90 | Adım          |
|---------|---------------|---------------|---------------|
| 0       | 0             | 0             | 0 (dönme yok) |
| 100     | 100           | 1             | 1 (90°)       |
| 450     | 90            | 1             | 1 (90°)       |
| -90     | 270           | 3             | 3 (270°)      |
| 340     | 340           | 4             | 4 % 4 = 0     |

`(derece + 45) / 90` tam sayı bölmesidir; küsurat atılır. 45 eklemek, "yarıyı geçtiyse bir üste yuvarla" demektir. Bu mantık `Stacker`'daki `donusDerecesi` ayarıyla aynı. `Stacker` ise ters yönden hesaplıyordu: fiziksel noktadan mantıksal noktaya. Ben  $8 \times 8$  noktanın hepsini, 4 dönüş ve ayna açık/kapalı durumları için bilgisayarda test ettim; iki kod her durumda aynı LED'i yakıyor.

## 5. Arka plan yenileme: Timer1 ve kesme (ISR)

Timer1, her 400 mikrosaniyede bir ana programı kısa süre durdurup `ISR` fonksiyonunu çalıştırır. Bu fonksiyon her çağrıldığında matrisin bir parçasını söndürür ve sıradaki parçayı yakar.

### Kesme (interrupt) nedir?

Ders çalışırken kapının çaldığını düşün: kitabı bırakırsın, kapıya bakarsın, sonra kaldığın sayfaya dönersin. Kesme de böyle çalışır. `loop()` ne yapıyorsa (`delay` içinde bekliyor olsa bile) işlemci onu durdurur, `ISR` 'yi çalıştırır, sonra kaldığı yerden devam eder. Ana program bunu fark etmez. `delay()` 'in serbest olmasının sebebi bu.

## Neden Timer1?

Arduino UNO'nun içinde kendi kendine sayan 3 zamanlayıcı var:

| Zamanlayıcı | Kim kullanıyor                                             | Bizim için               |
|-------------|------------------------------------------------------------|--------------------------|
| Timer0      | <code>millis()</code> , <code>delay()</code>               | Dokunursak zaman bozulur |
| Timer1      | Servo kütüphanesi, pin 9 ve 10'da <code>analogWrite</code> | Boş, biz aldık           |
| Timer2      | <code>tone()</code> (buzzer)                               | Dokunursak ses bozulur   |

Yani `matris.h` kullanan bir projede Servo kütüphanesi kullanılamaz. Pin 9 ve 10 zaten matrise bağlı olduğu için `analogWrite` sorunu yok.

## Timer1 ayarı: neden 400 µs?

```
noInterrupts();
TCCR1A = 0;
TCCR1B = (1 << WGM12) | (1 << CS11) | (1 << CS10);
TCNT1 = 0;
OCR1A = 99;
TIMSK1 |= (1 << OCIE1A);
interrupts();
```

Büyük harfli isimler işlemcinin içindeki ayar kutularıdır (register). Her biti bir ayar açar ya da kapatır; 3. bölümdeki `|=` ve `1 <<` mantığı burada da aynen geçerli.

| Satır                    | Anlamı                                           |
|--------------------------|--------------------------------------------------|
| <code>TCCR1A = 0</code>  | Önceki ayarları temizle                          |
| <code>WGM12</code>       | CTC modu: sayacı hedef sayıya gelince sıfırla    |
| <code>CS11   CS10</code> | Bölücü (prescaler) 64: 16 MHz'lik saati 64'e böl |
| <code>TCNT1 = 0</code>   | Sayacı sıfırdan başlat                           |
| <code>OCR1A = 99</code>  | Hedef sayı: 0'dan 99'a, yani 100 tık             |
| <code>OCIE1A</code>      | Hedefe ulaşıncaya kesme üret                     |

Hesap:

**16.000.000 / 64 = 250.000 tık/saniye → 1 tık = 4 µs → 100 tık = 400 µs**

Saniyede 1 / 0,0004 = 2500 kesme demek. Ayar sırasında yarım kalmış bir ayarla kesme gelmesin diye kod `noInterrupts()` ve `interrupts()` arasına alınır.

## Kesme fonksiyonu adım adım

Her satır 3 parçaya bölünür, toplam  $8 \times 3 = 24$  parça olur. `_parca` değişkeni 0'dan 23'e kadar sayar ve başa döner. Kesme her çalıştığında şunları yapar:

1. **Önce söndür.** Son yaktığı satırı `_yanikSatir`, sütunları `_yanikMaske` değişkeninde hatırlar. Sadece onları kapatır; 16 pinin hepsini kapatmaya gerek yok, böylece zaman kazanır.
2. **Sıradaki parçaya geç.** `_parca = (_parca + 1) % 24;` ile bir artırır, 24 olunca 0'a döner.
3. **Parçayı bul.** `r = _parca / 3` satır numarasıdır, `_parca % 3` parça numarasıdır. Örneğin `_parca = 7` ise `r = 2` (3. satır), `parça = 1` (sütun 3-5).
4. **Bu parçada ne yanacak?** `maske = _fiziksel[r] & _PARCA[parça]`.
5. **Boşsa atla.** Maske 0 ise bu parçada yanacak LED yoktur; bir sonraki parçaya bakar. En fazla 24 deneme yapar, ekran tamamen boşsa hiçbir şey yakmadan çıkar.
6. **Doluysa yak.** Maskedeki sütunları LOW, satırı HIGH yapar. Ne yaktığını hatırlar ve `return` ile çıkar. LED bir sonraki kesmeye kadar, yani 400  $\mu$ s yanar.

## Bu tasarımın sonuçları

- **Yenileme hızı:** Ekran tamamen doluyken  $24 \text{ parça} \times 400 \mu\text{s} = 9,6 \text{ ms'}$ de bir tur döner. Bu da saniyede yaklaşık 104 kez yenileme demek; göz titreme görmez.
- **Parlaklık farkı:** Boş parçalar atlandığı için tek bir LED sürekli yanar ve çok parlaktır. Ekran tamamen doluyken her LED zamanın sadece  $1/24$ 'ünde yanar ve daha sönük görünür. Stacker'da da aynı durum vardı; çoklamanın doğal sonucu.
- **İşlemci yükü:** Bir `digitalWrite` yaklaşık 4-5  $\mu$ s sürer, kesme en fazla 8 tane çağırır. Kesme 400  $\mu$ s'nin yaklaşık 40-50  $\mu$ s'sini kullanır; işlemcinin  $\sim 10\%$ 'u. Öğrenci kodu bunu fark etmez.
- **Önce söndür, sonra yak sırası neden önemli?** Tersini yapsaydık, kısa bir an için hem eski hem yeni satır açık kalırdı. O anda yanlış LED'ler hafifçe yanardı. Buna "hayalet görüntü" (ghosting) denir.

## 6. Öğrenci komutları nasıl çalışıyor?

Her komut kısa bir "sarmalayıcı"dır: zor işi 3-5. bölümdeki parçalar yapar, komut sadece doğru parçayı doğru sırayla çağırır.

`matrisBaslat(derece, ayna)`

Sırayla şunları yapar: 16 matris pinini OUTPUT yapar ve hepsini kapalı konuma getirir; butonu ve buzzer'ı ayarlar; dereceyi dönüş adımına çevirir (4. bölüm); son olarak Timer1'i başlatır (5. bölüm).

`int derece = 0` yazımı **varsayılan parametre** demektir. `matrisBaslat();` yazarsan derece 0 olur, `matrisBaslat(90);` yazarsan 90. Böylece öğrenci çoğu zaman parametre yazmak zorunda kalmaz.

Buton `INPUT_PULLUP` modunda: Arduino'nun içindeki direnç pini 5 V'a çeker. Butona basılmazken pin **HIGH** okunur. Basınca pin GND'ye bağlanır ve **LOW** okunur. Mantık ters gibi görünebilir; bu yüzden `butonaBasiliMi()` bu tersliği öğrenciden saklar. Dışarıdan direnç takmaya gerek kalmaması da bu modun avantajı.

`ledYak(x, y)` ve `ledSondur(x, y)`

İkisi de aynı iç fonksiyonu çağırır:

```
void _ledAyarla(int x, int y, bool yansin) {
    if (x < 0 || x > 7 || y < 0 || y > 7) return; // ekran dışı: yok say
    byte yeni[8];
    for (byte i = 0; i < 8; i++) yeni[i] = _ekran[i]; // kopya al
    if (yansin) yeni[y] |= (1 << x); // biti aç
    else yeni[y] &= ~(1 << x); // biti kapat
    _ekranaYaz(yeni); // tek seferde yaz
}
```

**İlk satırdaki sınır kontrolü çok önemli.** C++ dizinin sınırını kendisi kontrol etmez. Öğrenci `ledYak(8, 3)` ya da döngüde `x <= 8` yazarsa kod dizinin dışındaki belleğe yazar. Bu da başka bir değişkeni bozar ve "kod bazen saçma davranıyor" gibi bulunması çok zor hatalara yol açar. Kontrol sayesinde ekran dışındaki nokta sessizce yok sayılır. Bu sayede 2. projedeki gibi "nokta ekrandan çıktı" durumları güvenli olur.

`ledYanikMi(x, y)`

`_ekran[y] & (1 << x)` sonucunu döndürür (3. bölümdeki "oku" işlemi). Projelerde henüz kullanılmıyor. İleride "başka bir şeye çarptı mı?" kontrolü için işe yarar; örneğin bir yılın oyununda.

**resimCiz(RESIM)**

```
const char* KALP[8] = {
    "##..##.",    // i = 0 -> en üst satır (y = 7)
    ...
    "....."     // i = 7 -> en alt satır (y = 0)
};
```

`const char*` bir yazıyı (string) gösterir; `[8]` ise bu yazılardan 8 tane olduğunu söyler. `resim[i][x]`, i. yazının x. harfidir. Fonksiyon her harfe bakar, '#' gördüğü yerde `yeni[7 - i]` satırının x. bitini açar. `7 - i` olmasının sebebi: kodda yukarıdan aşağı yazdığımız resim, ekranda da yukarıdan aşağı görünsün. Kodda nasıl görüyorsan matriste de öyle çıkar.

Döngüdeki `resim[i][x] != '\0'` koşulu bir güvenlik önlemi. C'de her yazı görünmez bir `'\0'` karakteriyle biter. Öğrenci bir satıra 8'den az karakter yazarsa, fonksiyon yazının bittiği yerde durur ve yanlış belleği okumaz.

**butonaBasiliMi() ve butonaBasildi() arasındaki fark**

|                   | butonaBasiliMi()                         | butonaBasildi()                               |
|-------------------|------------------------------------------|-----------------------------------------------|
| Soru              | Buton şu an basılı mı?                   | Butona şimdi <b>yeni</b> mi basıldı?          |
| Basılı tuttuğunda | Sürekli true                             | Sadece ilk anda bir kez true                  |
| Nerede kullanılır | Basılı tuttukça bir şey olsun (5. proje) | Her basış bir kez sayılsın (6., 7., 8. proje) |

`butonaBasildi()` iki fikre dayanır:

```
bool butonaBasildi() {
    bool simdi = butonaBasiliMi();
    bool yeni = (simdi && !_butonOnceki && millis() - _sonBasis > 50);
    if (yeni) _sonBasis = millis();
    _butonOnceki = simdi;
    return yeni;
}
```

**1. Kenar algılama:** Fonksiyon bir önceki çağrıda butonun ne durumda olduğunu `_butonOnceki` değişkeninde hatırlar. "Şimdi basılı VE önceden basılı değildi" ise yeni bir basış vardır. Buton basılı kaldıkça önceki durum da "basılı" olacağı için tekrar sayılmaz.

**2. Zıplama önleme (debounce):** Mekanik bir buton basıldığı anda metal temaslar birkaç milisaniye boyunca titrer. Arduino bunu hızlı hızlı "basıldı-bırakıldı-basıldı" olarak okur. Son basıştan 50 ms geçmeden gelen basışı saymayarak bu titreşimi yok sayarız.

Dikkat: `_butonOnceki` tek bir değişken. Aynı `loop` içinde `butonaBasildi()` iki farklı yerde çağrılırsa, basışı ilk çağırın yer "yakalar", ikincisi hiç görmez. Öğrenciler 10. sınıf görevlerinde buna takılabilir. Çözüm: sonucu bir kez bir değişkene alıp (`bool basildi = butonaBasildi();`) sonra o değişkeni kullanmak.

### `bekleBasildiMi(ms)`

```
bool bekleBasildiMi(unsigned long ms) {
    unsigned long baslangic = millis();
    while (millis() - baslangic < ms) {
        if (butonaBasildi()) return true;    // basıldı: hemen çık
        delay(1);
    }
    return false;                          // süre doldu, basılmadı
}
```

`delay(200)` sırasında Arduino başka hiçbir şey yapmaz; o sırada gelen kısa bir basış kaçabilir. Bu fonksiyon da bekler ama beklediği süre boyunca her milisaniyede butona bakar. 8. projede noktanın hem adım adım ilerlemesi hem de basışın anında yakalanması bununla mümkün oluyor.

`millis()`, Arduino açıldığından beri geçen milisaniyedir. `millis() - baslangic` yazımı bilinçli bir seçim: `millis()` yaklaşık 49 günde bir taşıp sıfırdan başlar. Çıkarma şeklinde yazılınca bu taşma anında bile doğru sonuç verir. `millis() < baslangic + ms` şeklinde yazsaydık o anda hata verirdi.

### `bip(frekans, sure)`

`tone()` sesi başlatır ve hemen geri döner; ses arka planda Timer2 ile çalar. Arkasından gelen `delay(sure)` ise ses bitene kadar bekler. Böylece arka arkaya yazılan `bip` komutları üst üste binmez, melodi gibi sırayla çalar. Frekans ne kadar yüksekse ses o kadar incedir: 262 = Do, 523 = bir oktav üst Do, 1047 = iki oktav üst.

## 7. Projeler: mantık ve görev çözümleri

Her proje bir öncekine tek bir yeni kavram ekler. Aşağıda her proje için kodun nasıl düşünüldüğü, dikkat edilmesi gereken noktalar ve görevlerin çözüm fikirleri var. Çözümler tek yol değil; öğrenci farklı ama çalışan bir yol bulursa o da doğrudur.

## Proje 1 — İlk Işık

**Yeni kavram:** `setup()` bir kez, `loop()` sonsuza kadar çalışır.

Köşeler `setup()` içinde bir kez yakılır ve hep yanık kalır. Bunun sebebi, LED'in durumunun ekran hafızasında saklanması ve kesmenin onu sürekli yeniden çizmesi. Ortadaki LED ise `loop()` içinde olduğu için durmadan yanıp söner. `ledSondur(3, 4)` sadece o noktayı söndürür, köşelere dokunmaz.

### Görev çözümleri:

- Diğer köşeler: `ledYak(0, 7);` ve `ledYak(7, 0);` satırlarını `setup()` içine ekle.
- Tam orta: 8 çift sayı olduğu için **tam ortada tek bir LED yok**. Orta 4 LED'dir: (3,3), (4,3), (3,4), (4,4). Güzel bir tartışma sorusu: "Neden tam ortayı bulamıyoruz?"
- İki kat hız: iki `delay(500)` değerini `delay(250)` yap.
- Baş harf: Harfi önce kareli kâğıda çizmek, sonra her kareyi bir `ledYak` satırına çevirmek en kolayı. Bu, 3. projedeki döngülerin neden icat edildiğini de hissettirir.
- Polis çıkarı:

```
void loop() {
  ledYak(0, 4); ledSondur(7, 4);
  delay(300);
  ledSondur(0, 4); ledYak(7, 4);
  delay(300);
}
```

## Proje 2 — Yürüyen Nokta

**Yeni kavram:** Değişken ve `if`.

**En önemli nokta:** `int x = 0;` satırı `loop()` 'un **dışında**. `loop()` her turda en baştan başlar; eğer `int x = 0;` içeride olsaydı, her turda x tekrar 0 olur ve nokta hiç yürümezdi. Dışarıda tanımlanan değişken (global değişken) turlar arasında değerini korur. Öğrencilerin en sık yaptığı hatalardan biri bu; derste bilerek göstermeye değer.

Her tur şunu yapar: ekranı temizle → noktayı x'te yak → bekle → x'i 1 artır → 7'yi geçtiyse 0'a dön. `x = x + 1` matematikte imkansız bir eşitlik gibi görünür. Programlamada `=` "eşittir" değil, "sağdakini hesapla, soldaki kutuya koy" demektir.

### Görev çözümleri:

- En üst satır: `ledYak(x, 7);`
- Hız: `int bekleme = 50;`
- `ekraniTemizle()` kaldırılınca nokta arkasında iz bırakır ve satır dolar. Program LED'i söndürmeyi kendi kendine bilmez; ona ne söylersen onu yapar.

4. Dikey: `ledYak(3, x);` yazmak yeter. `x` artık `y` yerine geçiyor.

5. Duvardan sekme:

```
int x = 0;
int yon = 1;           // 1 = sağa, -1 = sola

void loop() {
  ekraniTemizle();
  ledYak(x, 3);
  delay(bekleme);
  x = x + yon;
  if (x == 7 || x == 0) { // || = "veya"
    yon = -yon;           // yönü tersine çevir
  }
}
```

6. Her turda bir satır yukarı: `int y = 0;` ekle, `if (x > 7)` bloğunun içine `y = y + 1;` `if (y > 7) y = 0;` yaz ve `ledYak(x, y)` kullan.

### Proje 3 — Çizgi Çiz

**Yeni kavram:** `for` döngüsü.

```
for (int x = 0; x < 8; x++) { ... }
// başlangıç   koşul   her turdan sonra
```

"`x`'i 0 yap. `x` 8'den küçük olduğu sürece süslü parantezin içini çalıştır, sonra `x`'i 1 artır." `x` sırayla 0, 1, ..., 7 olur; 8 olunca döngü biter. `x++`, `x = x + 1` 'in kısa yazımıdır.

Çaprazda (i, i) yazılır, çünkü çapraz üzerindeki her noktada `x` ile `y` eşittir. İç içe döngüde dış döngünün (`y`) her turu için iç döngü (`x`) baştan sona çalışır. Toplam  $8 \times 8 = 64$  kez `ledYak` çağrılır. Bu yüzden ekran satır satır dolar; daktilo gibi.

**Görev çözümleri:**

1. Sol sütun: `for (int y = 0; y < 8; y++) { ledYak(0, y); delay(100); }`

2. Diğer çapraz: `ledYak(i, 7 - i);`  $\rightarrow (0,7), (1,6) \dots (7,0)$ .

3. Çerçeve tek döngüyle de olur:

```
for (int i = 0; i < 8; i++) {
  ledYak(i, 0); ledYak(i, 7); // alt ve üst kenar
  ledYak(0, i); ledYak(7, i); // sol ve sağ kenar
}
```

4. Satranç tahtası: `for (int y = 0; y < 8; y++) for (int x = y % 2; x < 8; x = x + 2) ledYak(x, y);` — çift satırlarda x 0'dan, tek satırlarda 1'den başlar. `%` kalan demek: `y % 2` çift sayıda 0, tek sayıda 1 verir.
5. Tersten söndürme: `for (int y = 7; y >= 0; y--)` ve içerde `for (int x = 7; x >= 0; x--)` `ledSondur(x, y);`. **Tuzak:** Burada `int` yerine `byte` kullanılırsa döngü hiç bitmez. `byte` negatif olamaz: 0'dan sonra 1 azalınca 255 olur ve `>= 0` her zaman doğrudur.
6. Büyüyen kareler:  $k = 0..3$  için köşeleri  $(3 - k)$  ve  $(4 + k)$  olan karenin çerçevesini çiz (3. görevdeki çerçeve mantığı, 0 ve 7 yerine bu sınırlarla).

## Proje 4 — Emoji ve Animasyon

**Yeni kavram:** Dizi (liste) ve animasyon kareleri.

Her resim 8 yazıdan oluşan bir dizidir. Kodda nasıl görünüyorsa matriste öyle görünür (6. bölüm). İsimleri BÜYÜK harfle yazmak bir gelenektir: "bu değer program boyunca değişmez" demektir. `const` kelimesi de bunu derleyiciye söyler.

Animasyon, resimleri aralarında bekleyerek sırayla göstermektir; çizgi film gibi. Kalp atışında büyük kalp 60 ms bip + 300 ms bekleme, küçük kalp 400 ms görünür. Bu ritim "güm-güm" hissi verir. `bip` de beklediği için büyük kalp toplam 360 ms ekranda kalır.

### Görev çözümleri:

1. `BENIM_RESMIM` içindeki noktaları `#` ile değiştir, `loop()` içine `resimCiz(BENIM_RESMIM); delay(1000);` ekle. İpucu: önce kareli kâğıda çizdirmek çok işe yarar.
2. Daha hızlı: iki `delay` değerini küçült.
3. Farklı ses: `int nota = 500;` ekle, her turda `bip(nota, 60); nota = nota + 100; if (nota > 1500) nota = 500;`.
4. Göz kırpan yüz: `GÖZ_ACIK` ve `GÖZ_KAPALI` resimleri. Gerçekçi olması için göz uzun süre açık, kısa süre kapalı kalsın: `delay(2000)` ve `delay(150)`.
5. 4 kareli animasyon: 4 resim çiz, sırayla göster. Dönen çizgi kolay bir başlangıçtır: `|`, `/`, `-`, `\` şekilleri.
6. Hızlanan animasyon: `int hiz = 400;` global değişken; her turda `hiz = hiz - 20; if (hiz < 80) hiz = 400;` ve `delay(hiz)`.

## Proje 5 — Butonlu Yüz

**Yeni kavram:** Giriş (buton) ve `if / else`.

`loop()` saniyede binlerce kez döner. Her turda butona sorar ve cevaba göre bir resim çizer. Aynı resmi binlerce kez yeniden çizmek gereksiz bir iş, ama zararsız: `_ekranaYaz` resmi tek seferde değiştirdiği için titreme olmaz (3. bölüm). Bu proje öğrenciye "program butonu sürekli kontrol ediyor" fikrini verir.

### Görev çözümleri:

1. Ters: `MUTLU` ile `UZGUN` 'un yerini değiştir. Ya da koşulu `if (!butonaBasildi())` yap; `!` "değil" demektir.
2. Basılıyken ses: `if` bloğunun içine `tone(BUZZER, 1000);`, `else` bloğunun içine `noTone(BUZZER);` ekle. Burada `bip` kullanılırsa ses kesik kesik çıkar, çünkü `bip` her seferinde bekler.
3. Kendi resmi: 4. projedeki gibi yeni bir dizi çiz ve `UZGUN` yerine onu kullan.
4. Her basışta değişen yüz. Burada yeni bir kavram var: **durum (state)**. Program artık "şu an hangi yüzü gösteriyorum?" bilgisini aklında tutar:

```
bool mutluMu = true;

void loop() {
  if (butonaBasildi()) {
    mutluMu = !mutluMu;    // true ise false, false ise true yap
  }
  if (mutluMu) resimCiz(MUTLU);
  else        resimCiz(UZGUN);
}
```

5. Üç resim: `int sıra = 0;` → basışta `sıra = sıra + 1;` `if (sıra == 3) sıra = 0;` → sonra `if (sıra == 0) ... else if (sıra == 1) ... else ...` ile çiz. Bu, bir sonraki adımda "resim dizisi" fikrine hazırlık olur.

## Proje 6 — Tıklama Sayacı

**Yeni kavram:** Sayaç, bölme (`/`) ve kalan (`%`), Seri Monitör.

Tek bir sayıyı (0–63) ekrandaki bir noktaya çevirmenin püf noktası tam sayı bölmesidir. Arduino'da iki `int` bölününce küsurat atılır:  $13 / 8 = 1$  (1,625 değil). `%` ise kalanı verir:  $13 \% 8 = 5$ .

| sayac | $x = \text{sayac} \% 8$ | $y = \text{sayac} / 8$ | Ekranda         |
|-------|-------------------------|------------------------|-----------------|
| 0     | 0                       | 0                      | sol alt         |
| 7     | 7                       | 0                      | sağ alt         |
| 8     | 0                       | 1                      | 2. satırın başı |

| sayac | x = sayac % 8 | y = sayac / 8 | Ekranda          |
|-------|---------------|---------------|------------------|
| 13    | 5             | 1             | 2. satır, 6. LED |
| 63    | 7             | 7             | sağ üst          |

Sayaç tam olarak bir kitabı satır satır okumak gibi düşünülebilir: / kaçınıcı satırda olduğunu, % satırın kaçınıcı harfinde olduğunu söyler.

Kodda sıra önemli: önce mevcut sayac ile LED yakılır, sonra sayaç artırılır. Böylece ilk tıklama (0,0)'ı yakar. 64'e ulaşıncı 3 kez yanıp söner ve sıfırlanır.

Serial.begin(9600) , bilgisayarla 9600 baud hızında haberleşmeyi başlatır. Seri Monitör'deki hız da 9600 olmalı; farklıysa ekranda anlamsız karakterler görünür. print alt alta yazmaz, println yazdıktan sonra alt satıra geçer.

### Görev çözümleri:

1. Araçlar > Seri Port Ekranı, sağ alttan 9600 seç.
2. Satır dolunca farklı ses: sayaç artırıldıktan sonra if (sayac % 8 == 0) bip(2000, 150); .
3. 32'de dolsun: if (sayac == 32) . Ekranın sadece alt yarısı dolar.
4. 10 saniye oyunu:

```
unsigned long baslangic = 0;
bool oyunVar = false;

void loop() {
  if (butonaBasildi()) {
    if (!oyunVar) { // ilk basış oyunu başlatır
      oyunVar = true;
      baslangic = millis();
      sayac = 0;
      ekraniTemizle();
    }
    ledYak(sayac % 8, sayac / 8);
    sayac = sayac + 1;
  }
  if (oyunVar && millis() - baslangic > 10000) { // 10 sn doldu
    oyunVar = false;
    Serial.print("Skor: ");
    Serial.println(sayac);
    bip(300, 500);
  }
}
```

5. Yukarıdan aşağı: `int y = 7 - sayac / 8;` .

## Proje 7 — Elektronik Zar

**Yeni kavram:** Kendi fonksiyonunu yazmak ve `random` .

`buyukNokta(x, y)` dört `ledYak` satırını tek bir isim altında toplar. Parametreler (x, y) fonksiyonun "girdileri"dir; her çağrıda farklı yer verilir. Zarın yüzlerini çizerken 2×2'lik bir noktayı her seferinde 4 satırla yazmak yerine tek satırla çağırırız. Fonksiyonun asıl faydası bu: tekrarı azaltmak ve kodu okunur yapmak.

Noktalar 0, 3 ve 6. sütun/satırlardan başlar. Her nokta 2 LED genişliğinde olduğu için aralarında 1 LED boşluk kalır: 0-1 dolu, 2 boş, 3-4 dolu, 5 boş, 6-7 dolu. Orta nokta (3-4) tam ekranın ortasına denk gelir.

`random(1, 7)` 1 ile 6 arasında bir sayı verir. **Alt sınır dahil, üst sınır hariçtir**; 7 asla gelmez. Öğrenciler genelde `random(1, 6)` yazıp "neden hiç 6 gelmiyor?" diye sorar.

`randomSeed(micros())` neden var? Arduino'nun rastgele sayıları aslında sabit bir formülle üretilir. Bu yüzden Arduino her açıldığında aynı sayı sırasını verir. Butona basıldığı anın mikrosaniye cinsinden zamanı her seferinde farklıdır; bunu "tohum" olarak verince sıra da her seferinde farklı olur. Derste denemeye değer: bu satırı silip Arduino'yu birkaç kez resetle ve ilk zarı kaydet.

Yuvarlanma efektinde bekleme `50 + i * 20` ile 50 ms'den 230 ms'ye uzar, ses de `400 + i * 50` ile incilir. Gerçek bir zarın yavaşlayıp durması gibi görünür. Animasyondaki sayıların sonuçla bir ilgisi yok; sonuç en sonda ayrıca seçilir.

### Görev çözümleri:

1. Daha uzun dönsün: döngüde `i < 10` yerine `i < 20` .
2. 6 gelince melodi: `if (sonuc == 6) { bip(523, 100); bip(659, 100); bip(784, 300); }` .
3. Seri Monitör: `setup()` 'a `Serial.begin(9600);` , sonuçtan sonra `Serial.println(sonuc);` .
4. Kısa `zarCiz` :

```
void zarCiz(int sayi) {
    ekraniTemizle();
    if (sayi % 2 == 1) buyukNokta(ORTA, ORTA); // 1, 3, 5
    if (sayi >= 2) { buyukNokta(SOL, UST); buyukNokta(SAG, ALT); }
    if (sayi >= 4) { buyukNokta(SAG, UST); buyukNokta(SOL, ALT); }
    if (sayi == 6) { buyukNokta(SOL, ORTA); buyukNokta(SAG, ORTA); }
}
```

5. İki oyuncu: `int oyuncu = 1; int puan1 = 0; int puan2 = 0;` → sonuç çıkınca sırası gelen oyuncunun puanına ekle, Seri Monitöre yaz, `oyuncu = 3 - oyuncu;` ile sırayı değiştir ( $1 \leftrightarrow 2$ ). 20'yi geçen kazanır.
6. Hileli zar: `int sonuc = random(1, 10); if (sonuc > 6) sonuc = 6;` → 6 gelme ihtimali 1/6 yerine 4/9 olur. Olasılık konusunu konuşmak için güzel bir fırsat.

## Proje 8 — Ortaya Durdur

**Yeni kavram:** Öğrenilenleri birleştirip oyun yazmak.

Oyunun tüm durumu 4 değişkende: `x` (nokta nerede), `yon` (nereye gidiyor), `hiz` (ne kadar hızlı), `skor`. `loop()` 'un her turu oyunun bir "karesi"dir:

1. Ekranı temizle, hedefi ve noktayı çiz.
2. `hiz` kadar bekle, bu sırada butonu dinle (`bekleBasildiMi`).
3. Basıldıysa **karar ver**: nokta 3 ya da 4'teyse isabet. Değilse ıskalama. Sonucu göster, 1 saniye bekle, noktayı başa al.
4. Basılmadıysa **ilerle**: `x = x + yon;` . Duvara geldiyse yönü çevir.

`if / else` yapısı sayesinde bir turda ya karar verilir ya da ilerlenir, ikisi birden asla olmaz.

Her isabette `hiz` 20 ms azalır; oyun zorlaşır. **`if (hiz < 40) hiz = 40;` satırını neden şart?** Bu sınır olmasa 10 isabetten sonra `hiz` 0 olur, sonra da negatife düşer.

`bekleBasildiMi` parametreyi `unsigned long` (işaretsiz) olarak alır. -20 gibi negatif bir sayı işaretsiz çevrilince yaklaşık 4,29 milyar olur ve oyun ~49 gün donar. Sınır koymak programı bu tür hatalardan korur.

**Stacker ile bağlantı:** Stacker'daki `seviye0yna` fonksiyonunda bloğun kayıp butonla durduğu döngü tam olarak bu mantık. Stacker üstüne şunları ekliyor: blok genişliği, üst üste katlar, taşan kısmın kesilmesi ve seviyeler. Öğrenciler bu projeyi anladıktan sonra Stacker kodunu okumaya hazır olur.

### Görev çözümleri:

1. Hedefi sola kaydırmak için **iki yeri birlikte** değiştirmek gerekir: `hedefCiz()` içindeki sütunlar ve `if (x == 3 || x == 4)` koşulu. Biri unutulursa hedef bir yerde görünür, isabet başka yerde sayılır. Bu hata, "aynı sayıyı iki yere yazma, bir değişkene koy" kuralını öğretmek için birebir; 5. göreve de zemin hazırlar.
2. Hız: `int hiz = 300;` (başlangıç) ve `hiz = hiz - 10;` (daha yavaş zorlaşma). ıskalamada geri dönülen `hiz = 200;` değerini de güncellemeyi unutma.
3. 2 LED genişliğinde blok: `ledYak(x, 3); ledYak(x + 1, 3);` , duvar kontrolü `x == 0 || x == 6` , isabet `x == 3` (blok 3-4'ü tam kaplar).

4. Skoru göstermek: isabette `Serial.println(skor);`. İskalamada `skor = 0;` satırından **önce** 6. projedeki mantıkla `for (int i = 0; i < skor; i++) ledYak(i % 8, i / 8);` ve `delay(2000);`.
5. Rastgele hedef: `int hedef = 3;` global değişken. `hedefCiz()` içinde 3 ve 4 yerine `hedef` ve `hedef + 1`; koşul `x == hedef || x == hedef + 1`; isabetten sonra `hedef = random(1, 6);`.
- 6.3 can: `int can = 3;` → isalamada `can = can - 1;` → `if (can == 0) { resimCiz(OYUN_BITTI); delay(3000); can = 3; skor = 0; hiz = 200; }`. `OYUN_BITTI` resmini öğrenci kendisi çizer.

## 8. Öğrencilerin sorabileceği sorular

### Kod ve dil

**= ile == arasındaki fark ne?** `x = 3` x'e 3 koyar. `x == 3` "x 3'e eşit mi?" diye sorar. `if (x = 3)` yazılırsa derleyici çoğu zaman hata vermez: x'i 3 yapar ve koşul her zaman doğru sayılır. Projelerde en sık göreceğin hatalardan biri.

**Neden x ve y 0'dan başlıyor?** C++'ta diziler 0'dan numaralanır. 8 elemanlı bir dizinin elemanları 0-7'tür. Ekranı da buna uydurduk; böylece `for (int x = 0; x < 8; x++)` döngüsü tam olarak ekranı gezer.

### Hangi değişken tipini ne zaman kullanacağım?

| Tip                        | Ne tutar (Arduino UNO'da) | Nerede kullandık                           |
|----------------------------|---------------------------|--------------------------------------------|
| <code>bool</code>          | true / false              | <code>mutluMu</code> , buton durumu        |
| <code>byte</code>          | 0 ... 255                 | Ekran hafızası, pin numaraları             |
| <code>int</code>           | -32.768 ... 32.767        | Konum, sayaç, hız: öğrenci kodlarının çoğu |
| <code>unsigned long</code> | 0 ... ~4,29 milyar        | <code>millis()</code> ile zaman tutmak     |

Kural: Öğrenciler varsayılan olarak `int` kullansın. Zaman için `unsigned long` kullansınlar, çünkü `millis()` 32.767'yi yaklaşık 33 saniyede geçer.

**"expected ';' before ..." hatası ne demek?** Bir önceki satırın sonunda noktalı virgül unutulmuş. Hata mesajı genelde hatanın **bir altındaki** satırı gösterir; öğrencilere üst satıra bakmayı öğret.

**"'ledYak' was not declared in this scope" hatası?** En üstteki `#include "matris.h"` silinmiş ya da `.ino` dosyası `matris.h` olmadan başka bir klasöre kopyalanmış. Büyük/küçük harf de önemli: `ledyak` çalışmaz.

## Devre ve ekran

**Kod yüklendi ama hiç LED yanmıyor.** Sırayla kontrol et: `setup()` içinde `matrisBaslat();` var mı? 1. haftadaki `matris_test` kodu çalışıyor mu? Çalışmıyorsa sorun kabloda, çalışıyorsa koddadır.

**Resim yan ya da ters duruyor.** `matrisBaslat(90);`, `matrisBaslat(180);` ya da `matrisBaslat(270);` dene. Ayna gibi ters görünüyorsa `matrisBaslat(0, true);`.

**Ekran tamamen doluyken neden sönük, tek LED neden çok parlak?** Kesme boş parçaları atladığı için tek LED sürekli yanar. Ekran doluyken her LED zamanın sadece 1/24'ünde yanar (5. bölüm). Bu bir hata değil; çoklamanın doğası.

**Neden `delay` kullanınca ekran donmuyor? Stacker'da donuyordu.** Yenilemeyi artık ana program değil Timer1 kesmesi yapıyor. Kesme `delay` sırasında da çalışır (5. bölüm).

**Seri Monitörde anlamsız karakterler çıkıyor.** Monitörün sağ altındaki hız 9600 olmalı, `Serial.begin(9600)` ile aynı.

## Senin için (öğretmen)

**Projede Servo motor kullanabilir miyiz?** `matris.h` ile birlikte hayır. Servo kütüphanesi de Timer1'i kullanır ve ikisi çakışır (5. bölüm). Buzzer ( `tone` ) sorun değil, o Timer2'yi kullanıyor.

**`matris.h` 'de bir şey değiştirirsem?** Her proje klasöründe ayrı kopya olduğu için 8 dosyayı da ayrı ayrı güncellemek gerekir. İleride tek kopyada toplamak istersen `matris.h` 'yi bir Arduino kütüphanesine ( Belgeler/Arduino/libraries/Matris/ klasörü) çevirebiliriz. O zaman sketch'lerde `#include <matris.h>` yazmak yeter. Ama öğrenci bilgisayarlarına da kurulması gerekir; derste "klasörü kopyala, aç, yükle" yaklaşımı daha az sorun çıkarır.

**Bir öğrenci " `matris.h` 'nin içinde ne var?" diye sorarsa?** Kısa cevap: "Matris aslında aynı anda sadece 3 LED yakabiliyor. Bu dosya saniyede 2500 kez sıradaki 3 LED'e geçiyor, gözümüz de hepsini birden yanıyor sanıyor." Meraklı 10. sınıflar için Stacker kodundaki `ekraniBirKezCiz` fonksiyonu aynı fikrin kesmesiz, daha okunur bir hali; `matris.h` 'den önce onu göstermek daha kolay.